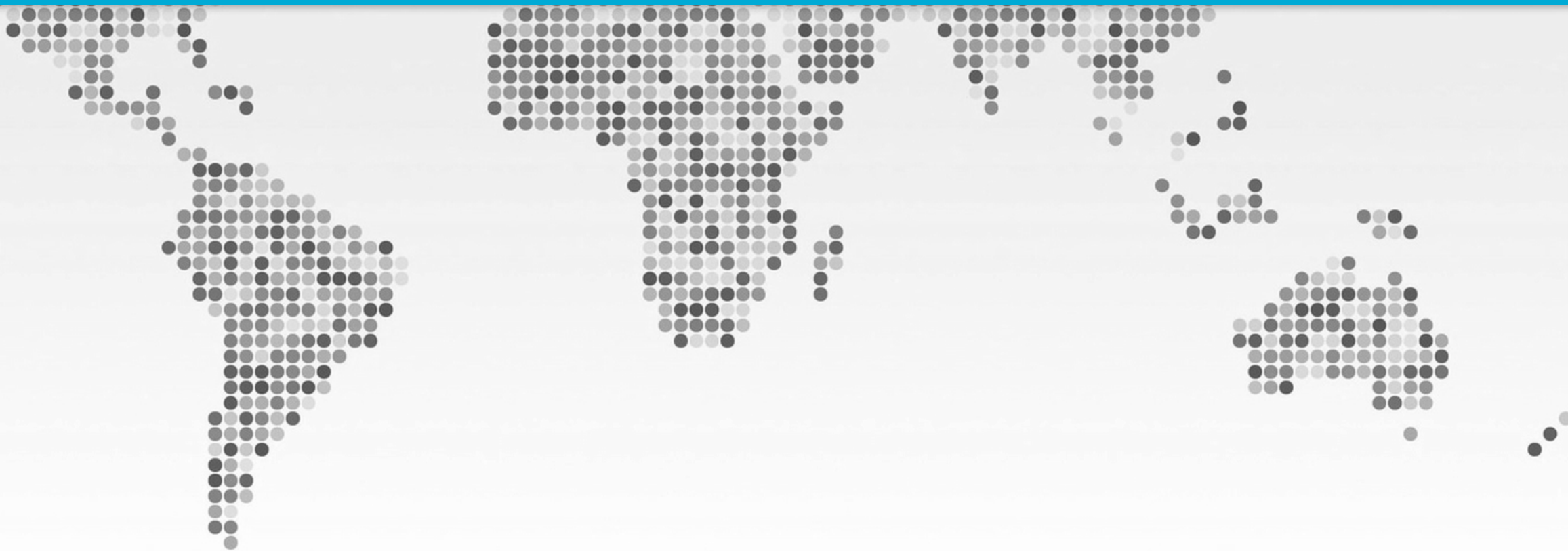


JUnit

Ingegneria del Software



Come verifichiamo la correttezza del seguente metodo?

```
public class Calcolatrice
{
    public int somma(int a, int b){
        return a + b;
    }
}
```

Come verifichiamo la correttezza del seguente metodo?

```
public class Calcolatrice
{
    public int somma(int a, int b){
        return a + b;
    }
}
```

Fino all'anno scorso qualcuno di voi avrebbe scritto qualcosa del genere

```
public class Main
{
    public static void main(String[] args) {
        Calcolatrice calc = new Calcolatrice();
        System.out.println(calc.somma(2,10));
        System.out.println(calc.somma(1,0));
        System.out.println(calc.somma(80,13));
    }
}
```

The logo for JUnit, featuring the word "JUnit" in a bold, sans-serif font. The "J" is bright green, and the "Unit" is red. Above the text is a faint, light gray rectangular box.

JUnit

Cos'è JUnit?

JUnit è un framework open-source che può essere utilizzato per scrivere ed eseguire e **rieseguire** i **test** in **automatico** .

N.B. Non fatevi ingannare dal nome! Con JUnit possono essere fatti sia i test di unità (che vedremo oggi) che test altri tipi di test!

Cos'è JUnit?

JUnit è un framework open-source che può essere utilizzato per scrivere ed eseguire e **rieseguire** i **test** in **automatico** .

N.B. Non fatevi ingannare dal nome! Con JUnit possono essere fatti sia i test di unità (che vedremo oggi) che test altri tipi di test!

Ogni framework utilizzato per scrivere casi di test ha diverse “best practices” che devono essere seguite:

- Ogni caso di test deve essere indipendente dagli altri
- Deve poter individuare e riportare gli errori per ogni caso di test
- Deve permettere di scrivere ed eseguire i casi di test in maniera semplice

JUnit configurazione

Uno dei metodi più semplici per utilizzare JUnit è quello di aggiungerlo come dipendenza all'interno del file **POM.XML** (per progetti con sistema di building MAVEN).

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.8.1</version>
  <scope>test</scope>
</dependency>
```

Se non hai idea di dove andare a pescare queste informazioni esistono **repository online** che non solo vi trovano la componente, ma vi **generano il codice** da inserire direttamente nel file pom.xml :

<https://mvnrepository.com/repos/central>

JUnit: Test Fixtures

JUnit offre molte funzionalità per permettere di scrivere facilmente casi di test, tra cui le text fixtures:

- Annotazioni che permettono di eseguire delle operazioni prima o dopo l'esecuzione di un caso di test, o di tutta la classe di test (JUnit 5).
 - **@BeforeEach**: Quando vogliamo eseguire una serie di operazioni **PRIMA** di ogni test
 - **@AfterEach**: Quando vogliamo eseguire una serie di operazioni **DOPO** ogni test
 - **@BeforeAll**: Quando vogliamo eseguire una serie di operazioni **PRIMA** di iniziare ad eseguire la test suite
 - **@AfterAll**: Quando vogliamo eseguire una serie di operazioni **DOPO** aver eseguito la test suite
- Una vasta gamma di “asserzioni” che possono essere utilizzati per verificare i risultati dei test scritti

Assert

Tra le tante asserzioni che JUnit ci offre, possiamo citare le seguenti:

Assert	Per cos'è usata
<code>assertEquals(a, b)</code>	Confronta se due oggetti sono uguali
<code>assertArrayEquals(a, b)</code>	Confronta se due array sono uguali
<code>assertTrue(a)</code>	Verifica se lo stato di quell'oggetto è true
<code>assertThrows(e)</code>	Verifica il lancio di un'eccezione all'esecuzione del test

N.B. ogni caso di test deve testare una e una sola cosa!

Quindi, è buona norma (esistono casi particolari) andare a definire un assert per ogni caso di test.

Facciamo un esempio:

Consideriamo di avere la classe `Calculator` con il metodo “`addition`”.

Questo metodo ha due parametri che sono gli addendi dell’addizione.

```
public class Calculator {  
    6 usages  
    double risultato;  
  
    1 usage  
    public double addition(double firstNumber, double secondNumber) {  
        risultato = firstNumber + secondNumber;  
        return risultato;  
    }  
}
```

Facciamo un esempio:

Creiamo quindi la classe **CalculatorTest**.

```
public class CalculatorTest {  
  
    5 usages  
    private Calculator calculator;  
    4 usages  
    private double firstNumber;  
    4 usages  
    private double secondNumber;  
    4 usages  
    private double result;  
  
    @BeforeEach  
    public void setUp() {  
        calculator = new Calculator();  
    }  
  
    @AfterEach  
    public void tearDown() {  
        calculator = null;  
    }  
}
```

Facciamo un esempio:

Creiamo quindi la classe **CalculatorTest**.

```
@Test
public void testAddition() {
    firstNumber = 10;
    secondNumber = 10;

    result = calculator.addition(firstNumber, secondNumber);
    assertEquals("expected: 20, result);
}
```

Best Practices (and Bad Practices...):

Per evitare di incorrere in problemi consideriamo due scenari di due persone che lavorano ai test.



Ada Assertelli: ha seguito il corso e tutte le lezioni di tutorato di IS e ora è diventata Software Quality Assurance Engineer presso una multinazionale.



Bilbo Verificatorelli: non ha voluto seguire il corso di IS e pensa che l'importante sia imparare i comandi di JUnit.

Best Practices (and Bad Practices...):

1) I test devono essere atomici e indipendenti.



```
@Test
public void testAddizione() {
    // ... setup
    int risultato = calcolatrice.addizione(3, 5);
    assertEquals(8, risultato);
}

@Test
public void testSottrazione() {
    // ... setup
    int risultato = calcolatrice.sottrazione(8, 3);
    assertEquals(5, risultato);
}
```



```
@Test
public void testAddizioneESottrazioneEqualcosaltro() {
    // Bilbo scrive un unico test per diverse operazioni e rende complesso il test
    int risultato1 = calcolatrice.addizione(3, 5);
    int risultato2 = calcolatrice.sottrazione(8, 3);
    assertEquals(8, risultato1);
    assertEquals(5, risultato2);
    assertEquals(13, risultato1+risultato2);
}
```

Best Practices (and Bad Practices...):

2) I nomi devono essere significativi.



```
@Test
public void testMoltiplicazioneConZero() {
    int risultato = calcolatrice.moltiplicazione(4, 0);
    assertEquals(0, risultato);
}
```



```
@Test
public void test1() {
    int risultato = calcolatrice.moltiplicazione(4, 0);
    assertEquals(0, risultato);
}
```

Best Practices (and Bad Practices...):

3) Usare i giusti “assert” per evitare casi di test complessi.



```
@Test
public void testDivisione() {
    double risultato = calcolatrice.divisione(10, 2);
    assertEquals(5.0, risultato, 0.001);
}
```



```
@Test
public void testDivisione() {
    double risultato = calcolatrice.divisione(10, 2);
    assertTrue(risultato == 5.0);
}
```

Best Practices (and Bad Practices...):

4) Utilizzare correttamente le test fixtures



```
@Before
public void setUp() {
    // ... inizializza oggetti necessari per i test
    calcolatrice = new Calcolatrice();
}

@After
public void tearDown() {
    // ... pulisci risorse
    calcolatrice = null;
}
```



```
@Before
public void setUp() {
}

@After
public void tearDown() {
}
```

Best Practices (and Bad Practices...):

5) Usare i giusti “assert” per le eccezioni.



```
@Test
public void testDivision() {
    Exception exception = assertThrows(ArithmeticException.class, () ->
        calculator.division(8, 0));
    assertEquals("division by zero", exception.getMessage());
}
```



```
@Test
public void testDivisionePerZero() {
    // ... setup
    try {
        calcolatrice.divisione(5, 0);
        fail("Dovrebbe lanciare un'eccezione ArithmeticException");
    } catch (ArithmeticException e) {
        // atteso
    }
}
```

JUnit

Laboratorio di Ingegneria del Software

